

Software Testing and Quality Assurance

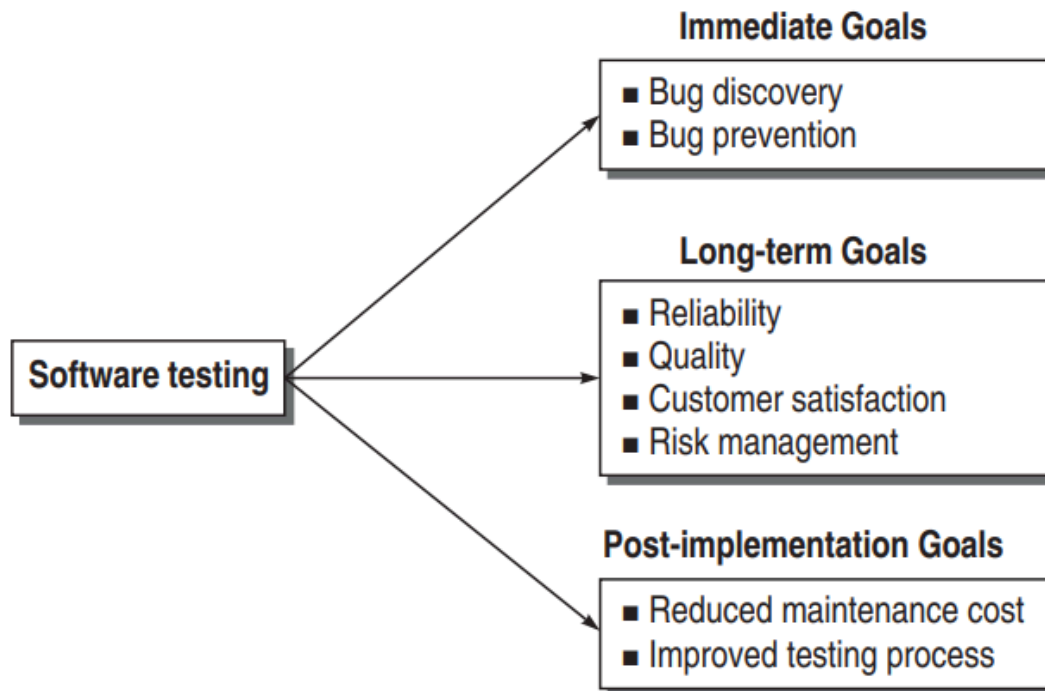
Module 1: Testing Methodology

Software Testing is the process of executing a program with the intent of finding errors.

Software testing is a systematic process of evaluating a software application or system to ensure it behaves as expected and meets specified requirements. It involves executing programs under controlled conditions with the intent of finding errors, defects, or areas that do not function according to design. Testing can focus on both functional aspects (does the software do what it's supposed to do?) and non-functional aspects (how well does it perform, is it secure, etc.). This process can be performed manually by humans or automatically using specialized tools. Software testing is essential for several critical reasons:

- **Detecting Defects Early:** Testing identifies bugs, errors, and flaws before the software reaches users, preventing costly and reputation-damaging failures post-release.
- **Ensuring Quality:** It helps guarantee that software meets both functional requirements and quality attributes like reliability, usability, performance, and security.
- **Reducing Costs:** Finding and fixing defects early in the development cycle is significantly cheaper and less disruptive than fixing them after deployment.
- **Risk Management:** Testing uncovers potential security vulnerabilities, performance issues, and other risks that could lead to system failure or data breaches.
- **Requirement Verification:** It confirms that the software fulfills both business and technical requirements, as well as complies with relevant standards and regulations.
- **User Satisfaction:** Thorough testing increases the likelihood that the software will work as intended for end-users, leading to higher adoption and satisfaction.
- **Maintaining Stability:** Testing ensures that new updates or changes do not break existing features, maintaining overall system stability and reliability.

Goals of Software Testing:



Immediate Goals

- Bug discovery: The immediate focus is to uncover defects (“bugs”) in the software, ideally as early as possible. This involves running the application with various test cases and closely observing its behavior to spot any inconsistencies with requirements.
- Bug prevention: By analyzing how bugs arise, development teams can update coding standards, improve developer education, and implement best practices (like code reviews and static analysis) to make similar mistakes less likely in future iterations. Prevention helps maintain code health and reduces the number of issues that reach testing stages.

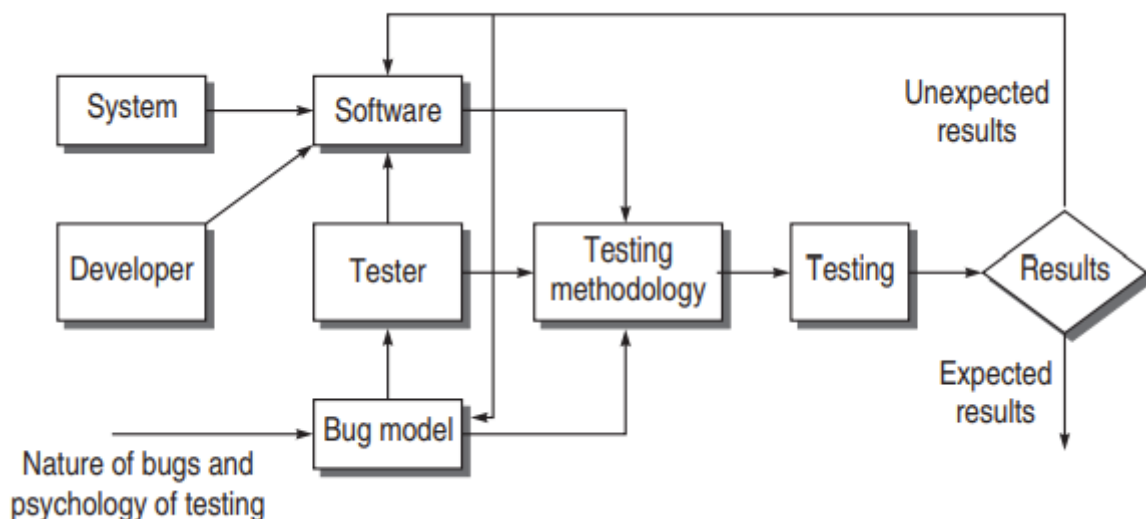
Long-term Goals

- Reliability: The software should perform its functions accurately and consistently, even under heavy load or over extended periods. Reliability is measured by metrics like mean time between failures (MTBF).
- Quality: This encompasses not only functional correctness but also security, performance, usability, maintainability, and scalability. Quality is assessed through standards compliance, user feedback, and defect rates.
- Customer satisfaction: Reliable, high-quality software with minimal defects leads to increased user satisfaction, better product reputation, and higher customer retention.
- Risk management: Testing identifies potential points of failure and helps teams prioritize fixes, reducing the likelihood of critical system failures and their business impact.

Post-implementation Goals

- Reduced maintenance cost: Early and thorough testing minimizes the number of defects that make it to production. This reduces the need for patches, hotfixes, and emergency releases, which are typically expensive and disruptive.
- Improved testing process: Post-release, teams review what went well and what didn't in their testing strategies. Lessons learned are applied to improve test coverage, automation, documentation, and communication for future projects.

Software Testing Model:



Software and Software Model:

Software is built after analysing the system in the environment. It is a complex entity which deals with environment, logic, programmer psychology, etc. But complex software makes it very difficult to test.

Since in this model of testing, our aim is to concentrate on the testing process, therefore the software under consideration should not be so complex such that it would not be tested. In fact, this is the point of consideration for developers who design the software. They should design and code the software such that it is testable at every point. Thus, the software to be tested may be modeled such that it is testable, avoiding unnecessary complexities.

Bug Model:

Bug model provides a perception of the kind of bugs expected. Considering the nature of all types of bugs, a bug model can be prepared that may help in deciding a testing strategy. However, every type of bug cannot be predicted. Therefore, if we get incorrect results, the bug model needs to be modified.

Testing methodology and Testing:

Based on the inputs from the software model and the bug model, testers can develop a testing methodology that incorporates both testing strategy and testing tactics. Testing strategy is the roadmap that gives us well-defined steps for the overall testing process. It prepares the planned steps based on the risk factors and the testing phase. Once the planned steps of the testing process are prepared, software testing techniques and testing tools can be applied within these steps. Thus, testing is performed on this methodology. However, if we don't get the required results, the testing plans must be checked and modified accordingly

Aspect	Exhaustive Testing	Effective Testing
Definition	Testing every possible input, path, and scenario exhaustively.	Testing key functionalities and critical areas within constraints.
Feasibility	Impractical for most real projects due to time and resource limits.	Practical and achievable within project limits.
Time & Cost	Very time-consuming and costly.	Time-efficient and cost-effective.
Approach	Theoretical and complete coverage attempt.	Prioritized, focused, and smart testing.
Scope	Covers all possible scenarios and combinations.	Covers important, high-impact test cases.
Use Cases	Suitable for small systems or where reliability is critical (e.g., healthcare, aerospace).	Suitable for most projects to ensure quality without exhaustive effort.

Software testing terminology

Failure. When the software is tested, failure is the first term being used. It means the inability of a system or component to perform a required function according to its specification. In other words, when results or behaviour of the system under test are different as compared to specified expectations, then failure exists.

Fault/defect/bug. Failure is the term which is used to describe the problems in a system on the output side, as shown below. Fault is a condition that actually causes a system to produce failure. Fault is synonymous with the words defect or bug. Therefore, fault is the reason embedded in any phase of SDLC

and results in failures. It can be said that failures are manifestations of bugs. One failure may be due to one or more bugs and one bug may cause one or more failures. Thus, when a bug is executed, then failures are generated. But this is not always true. Some bugs are hidden in the sense that these are not executed, as they do not get the required conditions in the system. So, hidden bugs may not always produce failures. They may execute only in certain rare conditions.

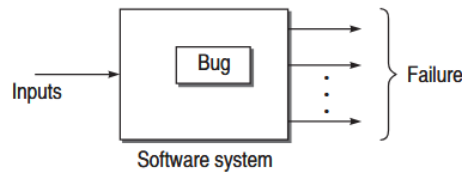
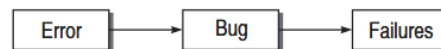
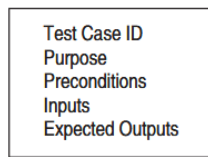


Figure 2.1 Testing terminology

Error. Whenever a development team member makes a mistake in any phase of SDLC, errors are produced. It might be a typographical error, a misunderstanding of a specification, a misunderstanding of what a subroutine does, and so on. Error is a very general term used for human mistakes. Thus, an error causes a bug and the bug in turn causes failures, as shown below.



Test case. Test case is a well-documented procedure designed to test the functionality of a feature in the system. A test case has an identity and is associated with a program behaviour. The primary purpose of designing a test case is to find errors in the system. For designing the test case, it needs to provide a set of inputs and its corresponding expected outputs.



- **Test case ID** is the identification number given to each test case.
- **Purpose** defines why the case is being designed.
- **Preconditions** for running the inputs in a system can be defined, if required, in a test case.
- **Inputs** should not be hypothetical. Actual inputs must be provided, instead of general inputs. For example, if two integer numbers have to be provided as input, then specifically mention them as 23 and 56.
- **Expected outputs** are the outputs which should be produced when there is no failure.

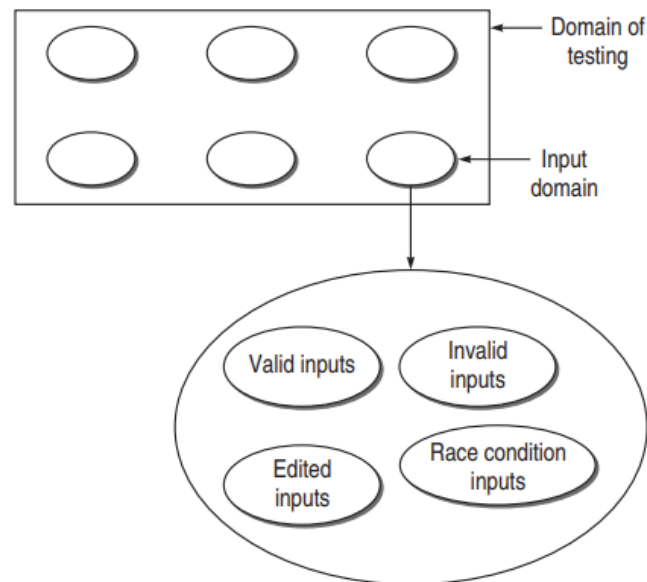
Test cases are designed based on the chosen testing techniques. They provide inputs when the system is executed. After execution, observed results are compared with expected outputs mentioned in the test case.

Testware. The documents created during testing activities are known as testware. Taking the analogy from software and hardware as a product, testware are the documents that a test engineer produces. It may include test plans, test specifications, test case design, test reports, etc. Testware documents should also be managed and updated like a software product.

Incident. When a failure occurs, it may or may not be readily apparent to the user. An incident is the symptom(s) associated with a failure that alerts the user about the occurrence of a failure.

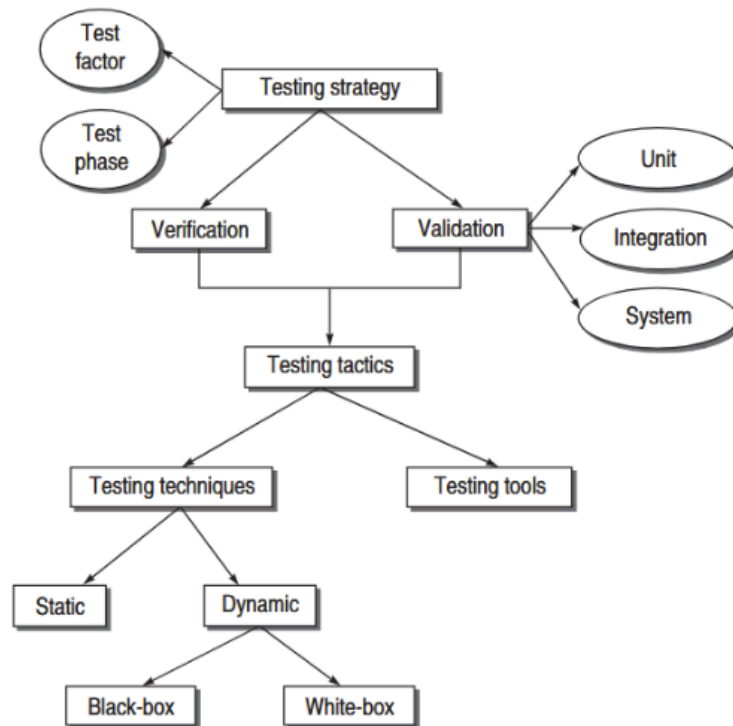
Test oracle. An oracle is the means to judge the success or failure of a test, i.e. to judge the correctness of the system for some test. The simplest oracle is comparing actual results with expected results by hand. This can be very time-consuming, so automated oracles are sought.

The domain of possible inputs in software testing refers to the complete set of all valid input values that a software program or system can accept. It is essentially the "input space" or range of inputs that the software is expected to handle correctly.



- **Valid Inputs:** Inputs that fully comply with the expected format, type, and constraints. The system is designed to accept and correctly process these.
- **Invalid Inputs:** Inputs that violate one or more constraints such as format, range, or type, and should be handled gracefully by the system (e.g., error message).
- **Edited Inputs:** Inputs that have been modified or corrected after initial entry, often to test how the system handles changes or updates to input data. This tests the system's ability to validate and process changed values.
- **Race Condition Inputs:** Inputs affected by timing issues when multiple processes or threads work with the same data simultaneously. Testing these inputs helps reveal concurrency problems where system behavior depends on unpredictable timing, leading to conflicts or unexpected results.

Testing Methodology:



Testing Strategy: The Foundation

The testing strategy shapes the overall what and why of testing, driven by:

- **Test Factors:** Key aspects such as functionality, performance, security, usability, etc.
- **Test Phases:** Different stages of the software development life cycle (SDLC) including component development, integration, and system completion.

It divides testing into:

- **Verification:** Focuses on building the product right by ensuring requirements and design specifications are correctly implemented. This often involves static testing like code reviews and inspections.
- **Validation:** Focuses on building the right product by executing the system to confirm it meets user needs, including dynamic testing at various levels.

Verification is ‘Are we building the product right?’

Validation is ‘Are we building the right product?’

Validation Levels: Ensuring Software Integrity - Validation testing is performed at multiple layers:

- **Unit Testing:** Testing individual software components or functions for correctness.
- **Integration Testing:** Examining interactions between combined units or modules.
- **System Testing:** Evaluating the complete, integrated system’s compliance with requirements.

Each level confirms the software’s behavior progressively from small units to the entire system.

Testing Tactics: The Execution Methods - Testing tactics answer how to carry out testing based on the strategy:

- **Testing Techniques**
 - **Static Testing:** Conducted without executing code, including reviews and static analysis to detect defects early.
 - **Dynamic Testing:** Testing by running the software.
 - **Black-box Testing:** Validates functionality against specifications without knowledge of internal code.
 - **White-box Testing:** Examines internal logic and structure—used to test code paths, conditions, and branches.
- **Testing Tools:** Automated and manual tools that aid testing efficiency, such as test management systems, automation frameworks, defect trackers, and performance testing tools.

Implementation in Software Development

- The methodology ensures testing is integrated into the development process, not isolated.
- It includes creating detailed test plans defining scope (features to test), coverage (test levels), test cases with expected outcomes, and resource allocation.
- Typical software testing methodologies commonly used include Waterfall, Agile, V-Model (Verification & Validation), Spiral, etc., each with unique approaches to integrate testing.

Test Strategy Matrix

A test strategy matrix identifies the concerns that will become the focus of test planning and execution. In this way, this matrix becomes an input to develop the testing strategy. The matrix is prepared using test factors and test phase.

Test Factors	Test Phase					
	Requirements	Design	Code	Unit test	Integration test	System test

Select and rank test factors

Based on the test factors list, the most appropriate factors according to specific systems are selected and ranked from the most significant to the least. These are the rows of the matrix.

Identify system development phases

Different phases according to the adopted development model are listed as columns of the matrix. These are called test phases.

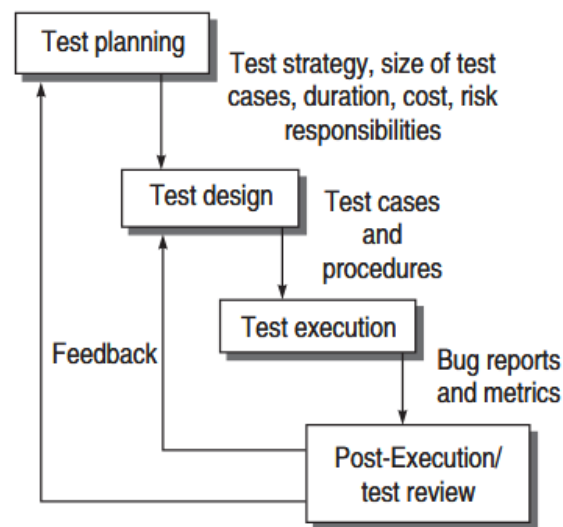
Identify risks associated with the system under development

In the horizontal column under each of the test phases, the test concern with the strategy used to address this concern is entered. The purpose is to identify the concerns that need to be addressed under a test phase. The risks may include any events, actions, or circumstances that may prevent the test program from being implemented or executed according to a schedule, such as late budget approvals, delayed arrival of test equipment, or late availability of the software application. As risks are identified, they must

be assessed for impact and then mitigated with strategies for overcoming them, because risks may be realized despite all precautions having been taken. The test team must carefully examine risks in order to derive effective test and mitigation strategies for a particular application. Concerns should be expressed as questions so that the test strategy becomes a high-level focus for testers when they reach the phase where it's most appropriate to address a concern.

STLC (Software Testing Life Cycle)

The testing process divided into a well-defined sequence of steps is termed as software testing life cycle (STLC). The major contribution of STLC is to involve the testers at early stages of development. This has a significant benefit in the project schedule and cost. The STLC also helps the management in measuring specific milestones.



Test planning

The goal of test planning is to take into account the important issues of testing strategy, viz. resources, schedules, responsibilities, risks, and priorities, as a roadmap. Test planning issues are in tune with the overall project planning. Broadly, following are the activities during test planning:

- Defining the test strategy.
- Estimate the number of test cases, their duration, and cost.
- Plan the resources like the manpower to test, tools required, documents required.
- Identifying areas of risks.
- Defining the test completion criteria.
- Identification of methodologies, techniques, and tools for various test cases.
- Identifying reporting procedures, bug classification, databases for testing, bug severity levels, and project metrics.

Based on the planning issues as discussed above, analysis is done for various testing activities. The major output of test planning is the test plan document. Test plans are developed for each level of testing. After analysing the issues, the following activities are performed:

- Develop a test case format.
- Develop test case plans according to every phase of SDLC.

- Identify test cases to be automated (if applicable).
- Prioritize the test cases according to their importance and criticality.
- Define areas of stress and performance testing.
- Plan the test cycles required for regression testing.

Test design

One of the major activities in testing is the design of test cases. However, this activity is not an intuitional process; rather it is a well-planned process. The test design is an important phase after test planning. It includes the following critical activities.

Determining the test objectives and their prioritization. This activity decides the broad categories of things to test. The test objectives reflect the fundamental elements that need to be tested to satisfy an objective. For this purpose, you need to gather reference materials like software requirements specification and design documentation. Then on the basis of reference materials, a team of experts compile a list of test objectives. This list should also be prioritized depending upon the scope and risk.

Preparing list of items to be tested. The objectives thus obtained are now converted into lists of items that are to be tested under an objective.

Mapping items to test cases. After making a list of items to be tested, there is a need to identify the test cases. A matrix can be created for this purpose, identifying which test case will be covered by which item. The existing test cases can also be used for this mapping. Thus it permits reusing the test cases. This matrix will help in:

- Identifying the major test scenarios.
- Identifying and reducing the redundant test cases.
- Identifying the absence of a test case for a particular objective and as a result, creating them.

Some attributes of a good test case are:

- A good test case is one that has been designed keeping in view the criticality and high-risk requirements in order to place a greater priority upon, and provide added depth for testing the most important functions.
- A good test case should be designed such that there is a high probability of finding an error.
- Test cases should not overlap or be redundant. Each test case should address a unique functionality, thereby not wasting time and resources.
- Although it is sometimes possible to combine a series of tests into one test case, a good test case should be designed with a modular approach so that there is no complexity and it can be reused and recombined to execute various functional paths. It also avoids masking of errors and duplication of test-creation efforts.
- A successful test case is one that has the highest probability of detecting an as-yet-undiscovered error.

Selection of test case design techniques. While designing test cases, there are two broad categories, namely black-box testing and white-box testing. Black-box test case design techniques generate test cases without knowing the internal working of a system. These will be discussed later in this chapter. The

techniques to design test cases are selected such that there is more coverage and the system detects more bugs.

Creating test cases and test data. The next step is to create test cases based on the testing objectives identified. The test cases mention the objective under which a test case is being designed, the inputs required, and the expected outputs. While giving input specifications, test data must also be chosen and specified with care, as this may lead to incorrect execution of test cases.

Setting up the test environment and supporting tools. The test created above needs some environment settings and tools, if applicable. So details like hardware configurations, testers, interfaces, operating systems, and manuals must be specified during this phase.

Creating test procedure specification. This is a description of how the test case will be run. It is in the form of sequenced steps. This procedure is actually used by the tester at the time of execution of test cases.

Test Execution

In this phase, all test cases are executed including verification and validation. Verification test cases are started at the end of each phase of SDLC. Validation test cases are started after the completion of a module. It is the decision of the test team to opt for automation or manual execution. Test results are documented in the test incident reports, test logs, testing status, and test summary reports.

Post-Execution/Test Review: A Post-Execution/Test Review in software testing is a critical phase that occurs after test execution is completed. This phase involves analyzing and reviewing the outcomes of the test cases that were executed to evaluate the software's quality and determine the next steps.

- **Test Execution Report:** This is a final summary document created by the test lead that details the stability and quality of the application. It includes:
 - Number of test cases written, executed, passed, and failed.
 - Pass/fail percentages.
 - Identification of defects or bugs.
 - Comments and severity levels for issues found.
 - This report helps decide if the software is ready to be handed over to customers or needs more fixes.
- **Outcome Analysis:** Test cases are analyzed for different statuses such as:
 - Passed: Software meets expectations.
 - Failed: Bugs are identified and reported.
 - Error: Issues in test execution itself.
 - Inconclusive: Unclear results needing further investigation.
- **Defect Reporting and Tracking:** Bugs found during execution are reported to developers, and the testing process may repeat after fixes.
- **Documentation and Communication:** The review includes sharing reports with stakeholders like development teams, management, and customers for transparency and collaborative improvement.

- **Test Closure Criteria and Lessons Learned:** The review might also formalize test closure if exit criteria are met (e.g., all critical defects resolved, test coverage completed). Often, a retrospective meeting synthesizes lessons learned to improve future projects.

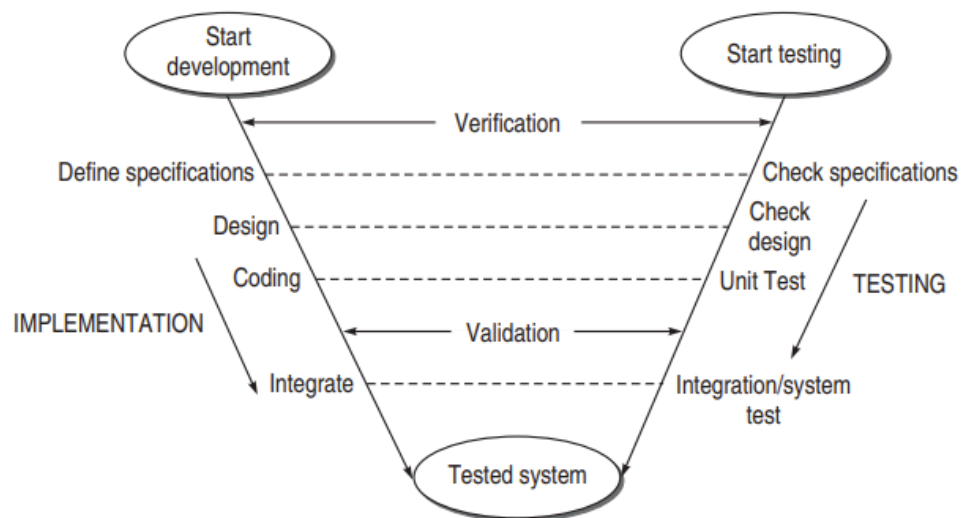
Overall, the Post-Execution/Test Review ensures thorough validation of the product, quality assessment, and helps refine testing processes for continuous improvement. It acts as an essential bridge between execution and release or further development phases.

Fundamental Testing Principles in Software Testing: There are several core principles that guide and underpin effective software testing. These principles help testers focus their efforts and ensure testing delivers value in identifying defects and improving software quality.

7 Key Principles of Software Testing

- **Testing Shows the Presence of Defects:** Testing can reveal defects but cannot prove the absence of defects. Even if no bugs are found, it doesn't guarantee the software is error-free.
- **Exhaustive Testing is Impossible:** Testing all possible inputs, scenarios, and combinations is not feasible except for trivial cases. Instead, risk-based and priority testing is used to cover the most critical areas effectively.
- **Early Testing Saves Time and Cost:** Testing activities should start as early as possible in the Software Development Life Cycle (SDLC), ideally during requirement analysis, to detect and fix defects sooner.
- **Defect Clustering:** A small number of modules or components often contain most of the defects. Focusing testing on those high-risk areas yields better returns.
- **Pesticide Paradox:** Running the same test cases repeatedly will eventually not find new bugs. Test cases need to be reviewed and updated regularly to detect different defects.
- **Testing is Context-Dependent:** Testing approaches and techniques vary depending on the type of software, its domain, and criticality (e.g., web vs. embedded systems).
- **Absence of Errors is a Fallacy:** Even if the software is bug-free, it may not meet the business needs or user expectations if it is not the right product.

V-Testing



The V Model (Verification and Validation Model) is a software development approach where testing activities are integrated with each phase of development, running both in parallel. Unlike the traditional Waterfall model, which delays testing until the end, the V Model incorporates testing throughout the process, starting as early as the requirements phase. This model ensures early detection of defects, lowering the cost and effort required to resolve issues and improving overall software quality.

Structure of the V Model in STLC: The V Model is visualized as a “V” shape:

- Left Side: Represents the development phases (Requirements, Design, Coding, etc.)
- Right Side: Corresponding testing phases are mapped to each development phase (Test Planning, Test Design, Unit/Integration/System Testing, Acceptance Testing, etc.)

Mapping of Development & Testing Activities

Development Phase	Corresponding Testing Activity
Requirement Analysis	Acceptance Testing
System Design	System Testing
Architecture/High-Level Design	Integration Testing
Module/Low-Level Design	Unit Testing
Coding/Implementation	Unit Testing

Key Advantages of V Model Testing in STLC

- Early defect identification: Testing starts early, so defects are caught quickly.
- Prevention of “Big Bang” testing: Testing is not left to the final stage, reducing risk.
- Improved cooperation: Promotes collaboration between developers and testers throughout the lifecycle.
- Structured quality assurance: Ensures software is validated and verified at every stage against requirements and standards.

Verification of Requirements:

In this type of verification, all the requirements gathered from the user’s viewpoint are verified. For this purpose, an acceptance criterion is prepared. An acceptance criterion defines the goals and requirements of the proposed system and acceptance limits for each of the goals and requirements. The acceptance criteria matter the most in case of real-time systems where performance is a critical issue in certain events. For example, if a real-time system has to process and make a decision for a weapon within x seconds, then any performance below this would lead to rejection of the system. Therefore, it can be concluded that acceptance criteria for a requirement must be defined by the designers of the system and should not be overlooked, as they can create problems while testing the system. The tester works in parallel by performing the following two tasks:

1. The tester reviews the acceptance criteria in terms of its completeness, clarity, and testability. Moreover, the tester understands the proposed system well in advance so that necessary resources can be planned for the project.

2. The tester prepares the Acceptance Test Plan which is referred at the time of Acceptance Testing

Functional design or high-level design

Functional design is the process of translating user requirements into a set of external interfaces. The output of the process is the functional design specification, which describes the product's behaviour as seen by an observer external to the product. The high-level design is prepared with SRS and software analysts convert the requirements into a usable product. In HLD, the software system architecture is prepared and broken into independent modules. Thus, an HLD document will contain the following items at a macro level:

1. Overall architecture diagrams along with technology details
2. Functionalities of the overall system with the set of external interfaces
3. List of modules
4. Brief functionality of each module
5. Interface relationship among modules including dependencies between modules, database tables identified along with key elements

High Level Design (HLD) Verification: Verification of HLD ensures that the proposed architecture, module structure, interfaces, and data design fulfill the specified requirements and are well-defined for subsequent development. This is critical because defects not caught here can propagate, causing complexity and increased costs later. Key Verification Activities in HLD:

- Architecture Evaluation: Ensure all major modules/components are identified and correctly structured.
- Data Design Review: Verify the customer/user's view of data models and high-level abstractions.
- Interface Design Review: Assess communications between modules and with external systems/users.
- Module Coupling & Cohesion: Check for strong cohesion within modules and loose coupling between them.
- SRS Traceability: Use a Requirement Traceability Matrix (RTM) to map requirements from the SRS to HLD artifacts.
- Formal Documentation: Reference the Software Design Document (SDD) for standards-compliance during verification.

Internal design or low-level design

Since HLD provides the macro-level details of a system, an HLD document cannot be given to programmers for coding. So the analysts prepare a micro-level design document called internal design or low-level design (LLD). This document describes each and every module in an elaborate manner, so that the programmer can directly code the program based on this. There may be at least one separate document for each module.

Low Level Design (LLD) Verification: LLD focuses on verifying detailed functionality of modules, internal logic, data structures, and interface specifications at a granular level. This phase ensures each component is ready for coding without ambiguity or gaps. Key Verification Activities in LLD:

- Detailed Module Review: Check that the operational details of each module match the intended design and requirements.

- **Data Structure & Logic Verification:** Validate algorithms, internal flows, and structure definitions as specified in the SDD.
- **Interface Detailing:** Ensure precise definition of internal and external interfaces for each module.
- **SRS & SDD Cross-check:** Maintain a two-way traceability matrix between SRS and individual module design to confirm that every requirement is mapped in the LLD.
- **Documentation Review:** Confirm every module's details are fully documented for development handoff.

Coding

If an LLD document is prepared for every module, then it is easy to code the module. Thus in this phase, using design document for a module, its coding is done. After understanding all the SDLC phases, we need to put together all the verification activities. As described earlier, verification activities are performed almost at every phase, therefore all the phases described above will be verified. Along with these verification activities performed at every step, the tester needs to prepare some test plans which will be used in validation activities performed after coding the system. When the coding is over for a unit or a system, and parallel verification activities have been performed, then the system can be validated. It means validation activities can be performed now. These are executed with the help of test plans prepared by the testers at every phase of SDLC.

Validation: Validation is a set of activities that ensures the software under consideration has been built right and is traceable to customer requirements. Validation testing is performed after the coding is over.

Validation Activities: The validation activities are divided into Validation Test Plan and Validation Test Execution which are described as follows:

Validation Test Plan: It starts as soon as the first output of SDLC, i.e. the SRS, is prepared. In every phase, the tester performs two parallel activities—verification at that phase and the corresponding validation test plan. For preparing a validation test plan, testers must follow the points described below.

- Testers must understand the current SDLC phase.
- Testers must study the relevant documents in the corresponding SDLC phase.
- On the basis of the understanding of SDLC phase and related documents, testers must prepare the related test plans which are used at the time of validation testing. Under test plans, they must prepare a sequence of test cases for validation testing.

The following test plans have been recognized which the testers have already prepared with the incremental progress of SDLC phases:

- **Acceptance test plan** This plan is prepared in the requirement phase according to the acceptance criteria prepared from the user feedback. This plan is used at the time of Acceptance Testing.
- **System test plan** This plan is prepared to verify the objectives specified in the SRS. Here, test cases are designed keeping in view how a complete integrated system will work or behave in different conditions. The plan is used at the time of System Testing.
- **Function test plan** This plan is prepared in the HLD phase. In this plan, test cases are designed such that all the interfaces and every type of functionality can be tested. The plan is used at the time of Function Testing.

- **Integration test plan** This plan is prepared to validate the integration of all the modules such that all their interdependencies are checked. It also validates whether the integration is in conformance to the whole system design. This plan is used at the time of Integration Testing.
- **Unit test plan** This plan is prepared in the LLD phase. It consists of a test plan of every module in the system separately. Unit test plan of every unit or module is designed such that every functionality related to individual units can be tested. This plan is used at the time of Unit Testing.

Validation test execution: Validation test execution can be divided in the testing activities:

Unit validation testing. The testing strategy is to first focus on the smaller building blocks of the full system. One unit or module is the basic building block of the whole software that can be tested for all its interfaces and functionality. Thus, unit testing is a process of testing the individual components of a system. A unit or module must be validated before integrating it with other modules. Unit validation is the first validation activity after the coding of one module is over. The motivation for unit validation as compared to the whole system are as follows:

- (a) Since the developer has his attention focused on a smaller building block of the software, i.e. unit or module, it is quite natural to test the unit first.
- (b) If the whole software is tested at once, then it is very difficult to trace the bug. Thus, debugging becomes easy with unit testing.
- (c) In large-scale projects, a number of modules may be there and each module may be composed of tens of thousands of lines of code. In such a case, it is not possible for a single developer to develop all the modules. There is a team of developers working on separate modules. Sometimes, some modules are sent to other organizations for development. This requires parallelism in software development. If we did not have the concept of modules, this type of parallelism would not have existed. Thus, every module can be developed and tested independently.

Integration testing. It is the process of combining and testing multiple components or modules together. The individual tested modules, when combined with other modules, are not tested for their interfaces. Therefore, they may contain bugs in an integrated environment. Thus, the intention here is to uncover the bugs that are present when unit tested modules are integrated.

Function testing. When the integrated system has been tested, all the specified functions and their external interfaces are tested on the software. Every functionality of the system specified in the functions is tested according to its external specifications. An external specification is a precise description of the software behaviour from the viewpoint of the outside world (e.g. user). Thus, function testing is to explore the bugs related to discrepancies between the actual system behaviour and its functional specifications. The objective of the function test is to measure the quality of the functional (business) components of the system. Tests verify that the system behaves correctly from the user/business perspective and functions according to the requirements, models, or any other design paradigm used to specify the application. The function test must determine if each component or business event:

- performs in accordance to the specifications,
- responds correctly to all the conditions that may be presented by incoming events/data,
- moves the data correctly from one business event to the next (including data stores), and
- is initiated in the order required to meet the business objectives of the system.

System testing. It is different from function testing, as it does not test every function. System testing is actually a series of different tests whose primary purpose is to fully exercise a computer-based system. System testing does not aim to test the specified function, but its intention is to test the whole system on various grounds where bugs may occur. For example, if the software fails in some conditions, how does it recover? Another example is protection from improper penetration, i.e. how secure the whole system is.

Acceptance testing. As explained earlier, in the requirement phase of SDLC, acceptance criteria for the system to be developed are mentioned in one contract by the customer. When the system is ready, it can be tested against the acceptance criteria contracted with the customer. This is called acceptance testing. Thus, acceptance testing can be defined as the process of comparing the final system with the needs of the customer as agreed on the acceptance criteria.